LABORATORY 1: Concurrency, Nondeterminism, and Race Conditions

---

1. Laboratory Objectives

By the end of this laboratory, students will be able to:

1. **plain** the difference between the illusion of parallelism (time-slicing) and true parallelism.
2. **Empirically demonstrate** the nondeterministic nature of the OS process scheduler.
3. **Identify** and reproduce a Race Condition in a controlled environment.
4. **Utilize** mutual exclusion mechanisms (pthread_mutex) to protect critical sections.
5. **Apply** dynamic analysis tools (ThreadSanitizer) to detect invisible concurrency bugs.

Lab resources:
https://drive.google.com/drive/folders/1909FGpzGND8d5d5oG2QfyzwDlhUJymFJ?usp=drive_link

---

2. Theoretical Background

2.1. From Sequential to Concurrent

In a sequential program, execution is deterministic: Statement A is followed by Statement B. In concurrent programming, the order of execution between different threads is **undefined**. The Operating System Scheduler may interrupt a thread at any moment to run another.

2.2. Anatomy of a Race Condition

A Race Condition occurs when:

1. Two or more threads access the same memory location simultaneously.
2. At least one thread writes (modifies) that memory.
3. There is no synchronization mechanism to enforce an ordering.

Classic Example: The count++ instruction.

Although it appears as a single line of C code, the CPU executes three distinct operations (Load-Modify-Store). If a Context Switch occurs between Load and Store, the update is lost.

2.3. The Critical Section and Mutex

The **Critical Section** is the segment of code that accesses a shared resource. This must be executed **atomically**.

A **Mutex** (Mutual Exclusion) acts as a lock/key:

- lock(): If the key is free, enter. If not, wait (block) until it becomes free.
- unlock(): Release the key for others.

---

3. Laboratory Activities

Prerequisites

- OS: Linux/macOS or Windows with WSL (Windows Subsystem for Linux).
- Compiler: gcc.

Exercise 1: The Illusion of Order

Goal: Demonstrate that the order of thread creation does not guarantee the order of execution.

1. Write a program that launches **30 threads**.
2. Pass a unique ID (0-29) to each thread upon creation.
3. The thread function should print: [Thread ID] started execution. and then [Thread ID] finished.
4. Run the program 5-10 times consecutively.
5. To make scheduling effects more visible, optionally synchronize thread start using a barrier (or run under CPU load).

Analysis Question:

> Do you observe a guaranteed order (0 to 29)? Do the output lines appear mixed or interleaved?

Exercise 2: "The Broken Counter" - Race Condition

Goal: Observe data corruption. This is the core of the lab.

1. Create a global variable volatile long counter = 0;.
2. Write a function void* increment(void* arg) that increments the variable **1,000,000 times** in a for loop.
3. In main, create **2 threads** that run this function simultaneously.
4. Wait for them to finish (pthread_join).
5. Print the final result.

Compilation: `gcc -o race race.c -pthread`

Analysis:

> The expected value is 2,000,000. What value did you actually get?
>
> Run objdump -d race | grep -A 10 increment (or similar). Identify the mov, add, mov instructions. Explain where the "window of vulnerability" lies.

Advanced Debugging (Google ThreadSanitizer):

Recompile the code using the sanitizer flag (standard in GCC/Clang):

- `gcc -o race_debug race.c -pthread -fsanitize=thread -g`
- `./race_debug`

Workaround (WSL/Kernel ASLR issue): If ThreadSanitizer fails at startup with `FATAL: ThreadSanitizer: unexpected memory mapping ...`, reduce ASLR entropy so TSan can reserve its shadow memory region:

```
sudo sysctl -w vm.mmap_rnd_bits=28
```

(Temporary setting—resets after reboot.)

Analyze the output. What type of error does TSan report? (e.g., Data race on ...).

Exercise 3: The Solution - Synchronization

Goal: Fix the bug using pthread_mutex_t.

1. Modify the code from Exercise 2.
2. Declare a global mutex: pthread_mutex_t lock;.
3. Initialize it in main (pthread_mutex_init).
4. Protect **only** the incrementation of counter using lock and unlock.
5. Measure the execution time (using time ./race in the terminal) compared to the previous version.

Analysis Question:

> Why is the program significantly slower now? Discuss *Context Switch Overhead* and *Serialization*.

---

4. Cheat Sheet: Local vs. Distributed Systems

Since this course lays the foundation for Distributed Systems, it is vital to understand the limitations of local concurrency models.

Feature	**Concurrent Programming (Local)**	**Distributed Systems**

Memory	Shared (Common Heap). Instant access.	No shared memory. Message passing only.
Synchronization	Mutex, Semaphores (OS Kernel).	Distributed Locks (e.g., Redis, ZooKeeper), Consensus (Raft/Paxos).
Time	Unique Global Clock (CPU Clock).	No global clock. (Clock Skew/Drift).
Failure	If the process crashes, everything stops.	Partial Failure: One node fails, others continue.
Latency	Nanoseconds.	Milliseconds (Network).

---

5. LabWork: "Bank Transfer & Deadlock"

Simulate a simple banking system with 2 accounts (A and B), each having its own Mutex.

1. **Thread 1** attempts to move money from A to B (Lock A -> Lock B).
2. **Thread 2** attempts to move money from B to A (Lock B -> Lock A).
3. Introduce a deliberate sleep(1) between the two lock acquisitions to force a **Deadlock**.
4. **Fix:** Solve the problem by imposing a **Global Locking Order** (always lock the account with the smaller ID first).
